AIOPS-03: Davis AI — Problems and Root Cause Analysis

> **Series:** AIOPS — Dynatrace Intelligence | **Notebook:** 3 of 8 | **Created:** May 2026 | **Last Updated:** 08/28/2026

Overview

Davis is the Causal AI engine. It watches the constant stream of anomaly signals (the raw `dt.davis.events`), groups related signals using Smartscape topology, and produces **problems** with a ranked root cause.

This notebook covers the Causal AI mechanism, the Problems app surface, the canonical DQL for querying problems, and the patterns for measuring detection quality.

Audience: SRE responding to incidents; platform admin tuning detection; observability lead reporting on MTTR.

Outcome: Working DQL for problem investigation, MTTR reporting, and root-cause analytics on your own data.

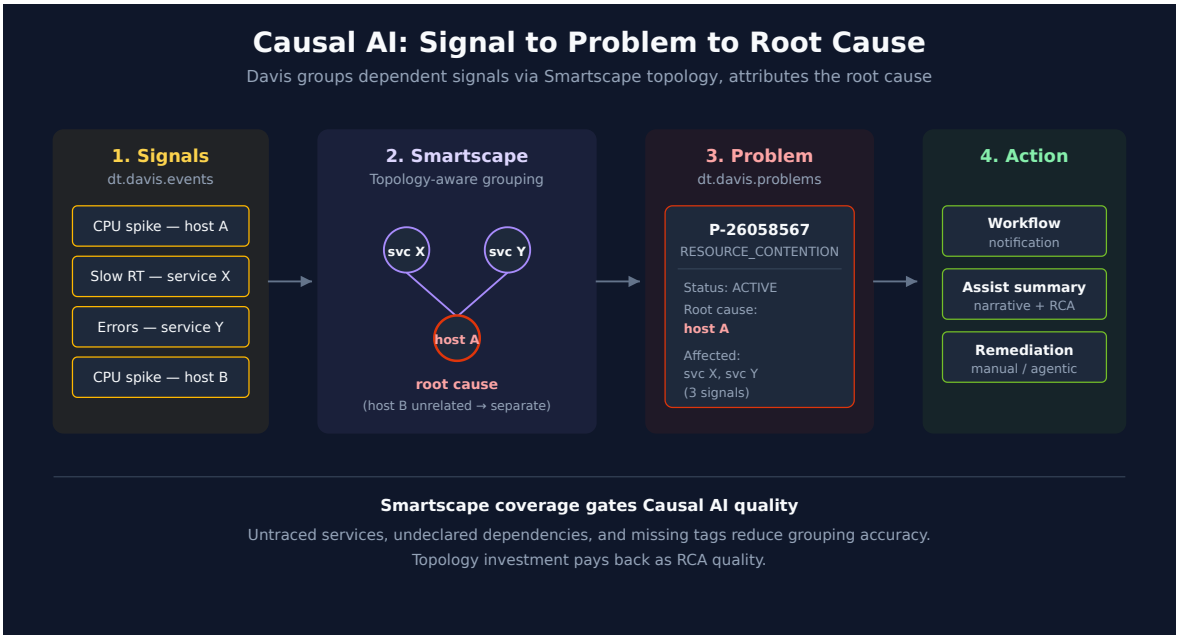

Table of Contents

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS Gen3 with Grail
Permissions	`events:read`, `storage:events:read`
Apps	Problems app
Topology	Smartscape entities for the systems you care about (Causal AI's accuracy correlates with topology completeness)

1. How Causal AI Groups Signals into Problems

Detectors fire constantly. Without grouping, every CPU spike, every slow request, every log error would page someone. The point of Causal AI is to ask: *which of these are the same incident?*

Davis answers using Smartscape — the dependency graph. Three signals on three different services that all depend on the same backing database go into one problem with

the database as root cause. Same three signals on unrelated services stay separate.

This is why Smartscape coverage matters. Causal AI's accuracy is bounded by topology completeness. Untraced services, undeclared dependencies, missing tags — all reduce the graph quality and produce worse grouping.

Causal AI is deterministic, not statistical. It walks the topology graph; it does not run an LLM or a black-box correlation. Two operators looking at the same data get the same root cause attribution.

The correlation keys

Grouping is not a black box you have to take on faith — it runs on fields you can query.

The universal rule is `dt.smartscape_source.id`. Most Davis events carry this field — **84.9% on a validation tenant over 7 days, 08/11/2026** (186,914 of 220,206) — holding the Smartscape entity ID of whatever the signal is *about*. It is the rule that is universal, not the field's presence: the remaining share is exactly the population this section is about. Events naming the same entity within the correlation timeframe collapse into a single problem. The semantic dictionary types it `smartscapeId` at stability `stable` — it holds an entity ID, not a display name.

This is the field a custom alert most often gets wrong, and the failure is silent — but the symptom is the opposite of the one usually expected. Leaving it unset does *not* leave the event unattributed. The events ingest API is explicit: when `entitySelector` is not set, "the event is associated with the environment (`dt.entity.environment`) entity." The event still arrives carrying `affected_entity_ids`; that array just names the environment instead of the thing that broke.

So these events do not fail to merge. **They over-merge.** Because every one of them names the same single entity, the grouping rule above does exactly what it is designed to do and collapses them together — gluing unrelated alerts into one problem. Measured on a validation tenant over 7 days on 08/11/2026: events falling back to the environment entity produced **28,004 firings across just 48 correlations — 583 firings per correlation, all naming one entity**. Every other event in the same window — 192,199 firings naming 1,207 real entities — ran at **1.1 firings per correlation**. That is the contrast: correctly attributed signals stay roughly one-to-one with the problems they raise, while the environment-fallback population collapses by more than five hundred to one. One detector alone fired 8,005 times and landed in a **single** correlation.

The cost is therefore lost attribution and unrelated alerts welded together, not an inflated problem count. The symptom to hunt is a problem with an implausibly broad, unrelated

blast radius and no usable root cause — not a problem-count spike. The remedy is unchanged: set the event template's entity to a real host, service, or workload ID. AIOPS-02 §4 covers setting it in the detector's event template; AIOPS-02 §8 has a query that finds these detectors in your own tenant.

Topology rules merge across entity types. Beyond exact-entity matching, Davis merges signals from entities in a known structural relationship — a process and the host it runs on are not two separate incidents:

Grouping field	Entity types merged into one problem
`dt.smartscape.host`	`HOST`, `PROCESS`, `CONTAINER`, `DISK`, `NETWORK_INTERFACE`
`dt.smartscape.container`	`CONTAINER`, `PROCESS`
`dt.smartscape.process`	`SERVICE`, `PROCESS`
`dt.smartscape.k8s_pod`	`K8S_POD`, `CONTAINER`
`dt.smartscape.k8s_node`	`K8S_NODE`, `K8S_POD`, `CONTAINER`
`dt.smartscape.k8s_deployment`	`K8S_DEPLOYMENT`, `SERVICE`
`dt.smartscape.k8s_replicaset`	`K8S_REPLICASET`, `SERVICE`
`dt.smartscape.k8s_statefulset`	`K8S_STATEFULSET`, `SERVICE`
`dt.smartscape.k8s_daemonset`	`K8S_DAEMONSET`, `SERVICE`
`dt.smartscape.k8s_job`	`K8S_JOB`, `SERVICE`

The table names the **grouping field**, not just a label, because that is what you filter on when you are working out why two alerts merged. Note the documentation introduces these as *"some of the examples of the grouping fields and potential entity types that can be merged"* — it is not an exhaustive list, so treat an unlisted merge as plausible rather than as a bug.

Three further deduplication layers run on top: time-based deduplication, so the same signal recurring inside the correlation window does not re-open a problem; causal merging across the dependency graph described above; and **frequent issue detection**, where Davis recognises a chronically recurring condition and stops raising it as a new problem on the reasoning that a permanent known issue is not news.

That last one is worth knowing before you conclude a detector has broken. A detector that "stopped alerting" may simply have had its condition classified as a frequent issue — check the event stream (`dt.davis.events`) rather than the problem feed to tell the two apart.

> **Sources:** Avoid overalerting (DT docs) — all ten grouping fields, introduced as examples rather than an exhaustive list, Dynatrace Intelligence (DT docs), Ingest an event — POST /api/v2/events/ingest (DT docs) — "If not set, the event is associated with the environment (`dt.entity.environment`) entity." **Derived:** the over-merge characterization combines that documented environment fallback with the same-entity grouping rule and the 08/11/2026 tenant measurement; the "check `dt.davis.events` rather than the problem feed" diagnostic follows from frequent-issue suppression acting at the event-to-problem step, combined with the two-data-object split in §3.

2. Problem Lifecycle and Fields

Status values: `ACTIVE` and `CLOSED` . (Not `OPEN` / `RESOLVED` — common mistake.)

Categories: `ERROR` , `RESOURCE_CONTENTION` , `AVAILABILITY` , `SLOWDOWN` , `CUSTOM_ALERT` . Categories are stable; counts vary widely by environment.

Severity is a separate axis from category. The unified `event.severity` field is an integer 1–5 (ITIL-aligned) that propagates from the constituent alerts up to the parent problem — see the field table below. **It is typed `experimental` in the semantic dictionary** (verified 08/27/2026), while `event.category` and `event.kind` are `stable` : fine for triage and reporting, but do not hard-code an integration or an SLA contract against it without a fallback.

Key fields on a problem record:

Field	Description
`event.id`	Internal problem ID
`display_id`	Human-readable ID like `P-26058567`
`event.name`	Short problem title
`event.description`	Longer narrative

Field	Description
`event.category`	One of the categories above
`event.severity`	Unified severity as an integer 1–5 (1=Critical, 2=High, 3=Medium, 4=Low, 5=Informational), aligned to the ITIL severity model. Propagates from the constituent alerts up to the parent problem. Coexists with `event.category` — severity answers *how bad*, category answers *what kind*.
`event.status`	`ACTIVE` or `CLOSED`
`event.start` / `event.end`	Timestamps; `event.end` is null on active problems
`root_cause_entity_id` / `root_cause_entity_name`	Davis-attributed root cause
`affected_entity_ids`	Array of impacted entity IDs
`dt.davis.event_ids`	Array of constituent signal IDs from `dt.davis.events`

Custom string-typed fields can be propagated onto problems via Settings — useful for team / alerting-profile / service-tier attribution. Numeric custom fields are not supported.

> **Changed (SaaS 1.346 — staged rollout from 08/25/2026): propagated problem fields keep only the latest value.** Verbatim: *"Propagated problem fields now store only the most recent event field value—they don't accumulate all historical values of an event field (for example, `trace_id`), rather they save only the latest value."*
>
> This changes what a propagated field *means*, which matters more than it first appears. A field like `trace_id` on a long-running problem previously accumulated every value the underlying events carried; now it holds one — the most recent. Two consequences for anything you have built on those fields:
>
> - **Do not treat a propagated field as a complete history.** A query that pulled `trace_id` off a problem to enumerate every affected trace will, after this change, return the last one only. Where you need the full set, go to the **events** (§3) rather than the problem record — the events are where the history actually lives, and that distinction is now load-bearing rather than stylistic.
> - **Comparisons across the rollout window are not like-for-like.** A problem that opened before the change and persisted through it may show accumulated values; one opened after will not.

> The upside is that a propagated field is now unambiguous — one value, the newest — rather than a list whose length depended on how long the problem stayed open.

3. The Two Data Objects: `dt.davis.problems` vs. `dt.davis.events`

Sibling streams in Grail. **Use the right one.**

Data object	Contains	event.kind	Use when you want
`dt.davis.problems`	Causal-AI-grouped problems	`DAVIS_PROBLEM`	The problem feed, MTTR, severity rollups
`dt.davis.events`	Raw, ungrouped signals	`DAVIS_EVENT`	Signal-level investigation, custom alert volume

> ⚠️ Some docs and tutorials show `fetch dt.davis.events | filter event.kind == "DAVIS_PROBLEM"`. It returns zero rows — and the reason matters: this is a **filter on the wrong table, not a deprecated form**. `dt.davis.events` has only ever carried `DAVIS_EVENT`, so there is no older behavior to migrate from and nothing in your tenant to fix. Use `fetch dt.davis.problems` for problems. Verified 08/27/2026 (7 days): 206,023 rows in `dt.davis.events`, all `DAVIS_EVENT`; 0 when filtered to `DAVIS_PROBLEM`; control `dt.davis.problems` → 2,774.

4. Active Problem Feed

The most-used query in the series — what's broken right now, ranked by start time.

```
// Active problems right now — investigation feed
fetch dt.davis.problems, from:-2h
| filter event.status == "ACTIVE"
| sort event.start desc
| fields display_id, event.name, event.category, event.start,
         root_cause_entity_name, affected_entity_ids
| limit 50
```

Filter by entity context to scope to a team or environment. Two common variants:

```
// Active problems in a specific Kubernetes namespace
fetch dt.davis.problems, from:-2h
| filter event.status == "ACTIVE"
| filter k8s.namespace.name == "production"
| sort event.start desc
| fields display_id, event.name, event.category, root_cause_entity_name
| limit 50
```

```
// Active problems for a specific host group
fetch dt.davis.problems, from:-2h
| filter event.status == "ACTIVE"
| filter dt.host_group.id == "HOST_GROUP-XXXXXXXX"
| sort event.start desc
| fields display_id, event.name, event.category, root_cause_entity_name
| limit 50
```

5. Problem Severity and Category Rollups

Two different axes, both useful in weekly operational reviews and for trending detection volume month over month:

- **Severity** (`event.severity` , integer 1–5) — *how bad* — for prioritization, SLA reporting, and routing. 1=Critical, 2=High, 3=Medium, 4=Low, 5=Informational (ITIL-aligned). `experimental` **stability** (08/27/2026) — usable, but pin a fallback before routing on it. Expect a narrow spread in practice: on the validation tenant 2,773 of 2,774 problems over 7 days were severity 3, so a one- or two-row rollup is the normal result, not a broken query.
- **Category** (`event.category`) — *what kind* — for spotting which failure modes dominate.

Start with the severity rollup, then break down by category.

```
// Last 7 days — problem count by severity (1=Critical … 5=Informational)
fetch dt.davis.problems, from:-7d
| fieldsAdd severity_label = if(event.severity == 1, "Critical",
    else: if(event.severity == 2, "High",
    else: if(event.severity == 3, "Medium",
    else: if(event.severity == 4, "Low",
    else: "Informational"))))
| summarize problem_count = count(), by:{event.severity, severity_label}
| sort event.severity asc
```

```
// Last 7 days — problem count by category
fetch dt.davis.problems, from:-7d
| summarize problem_count = count(), by:{event.category}
| sort problem_count desc
```

```
// Last 30 days — daily problem volume
fetch dt.davis.problems, from:-30d
| makeTimeseries problems_per_day = count(), interval:1d, by:{event.category}
```

6. MTTR by Category and Service

Mean Time To Resolution — how long does Davis say a problem stayed `ACTIVE` before it transitioned to `CLOSED`. Compute it as `event.end - event.start` over closed problems.

Reporting principle: prefer **median** and **p95** over arithmetic mean. A single long-running availability problem (a host that was forgotten on the network) skews the average wildly.

```
// MTTR by category over last 30 days
fetch dt.davis.problems, from:-30d
| filter event.status == "CLOSED"
| fieldsAdd duration_hours = (event.end - event.start) / 1h
| summarize {
    median_mttr_hours = percentile(duration_hours, 50),
    p95_mttr_hours    = percentile(duration_hours, 95),
    problem_count = count()
  },
  by:{event.category}
| sort problem_count desc
```

7. Root Cause Entity Distribution

Which entities are most often attributed as root cause? A heavy concentration on a small set of services usually means either (a) you have real recurring instability there, or (b) topology is incomplete and Davis keeps falling back to the same nodes.

```
// Top 20 root cause entities — last 7 days
fetch dt.davis.problems, from:-7d
| filter isNotNull(root_cause_entity_name)
| summarize problem_count = count(), by:{root_cause_entity_name,
root_cause_entity_id}
| sort problem_count desc
| limit 20
```

8. When No Root Cause Is Expected

`root_cause_entity_name` empty on a problem is not automatically a detection gap. §1 already establishes the mechanism: Causal AI is a deterministic graph walk over Smartscape topology — it explains one signal by finding another signal, on a *connected* entity, that a dependency edge plausibly links to it. When there's no second signal to reach, or no edge connecting the signals that did fire, the walk has nothing to traverse. An empty root cause is that walk terminating correctly, not failing.

In community troubleshooting experience, three conditions account for most "expected empty" cases:

- **Single-event problems.** The constituent-event count is one. With nothing else in the window to correlate against, there's no second point for a chain to reach — Davis still names the affected entity, but there is nothing to name as its cause.
- **No topology edge between co-occurring failures.** Multiple signals fired in the same window, but Smartscape has no dependency relationship between the entities they're on — common with custom entities, unmodeled network paths, or topology that hasn't finished discovering a newer service. Multiple events without a connecting edge still produce no chain, because the walk needs both a second event *and* a path to it.
- **Infrastructure-level, single-entity impact.** The failing entity and the affected entity are the same node — a workload stuck in a bad state with nothing upstream feeding it. The fault is the entity itself, not something that happened to it, so there's nothing else in the graph to name.

None of these three call for action. They're worth distinguishing from a fourth case that does:

- **A real topology gap.** Multiple related entities were affected and a dependency chain plausibly exists between them, but Smartscape's model doesn't capture the edge — an undeclared dependency, a missing trace-context propagation hop, an entity type Smartscape doesn't yet model relationships for. This is the case where the topology-completeness work §1 already calls out genuinely improves the next problem's RCA.

A practical way to tell them apart in your own data: a null-root-cause problem with a single affected entity sits in the "expected empty" population; a null-root-cause problem with *multiple* affected entities is the one worth auditing for a topology gap.

Setting this expectation with stakeholders matters as much as the mechanism itself — a target of 100% RCA coverage treats every empty field as a miss, when a meaningful share of them are Causal AI doing exactly what it's designed to do.

> **Derived:** the "expected empty" categories above follow from applying §1's deterministic graph-walk mechanism (no chain forms without a second event *and* a connecting Smartscape edge) to the null-root-cause case — not a single-source Dynatrace claim.

Track this as a trend, not a snapshot. The bucketing query above characterizes a point in time; the same `root_cause_entity_id` field trended daily shows whether the expected-empty population is holding steady or whether something upstream — topology, tagging, correlation — is regressing.

```
// Characterize your own null-root-cause problems: single-entity (expected) vs multi-entity (audit for topology gap)
fetch dt.davis.problems, from:-30d
| filter isNull(root_cause_entity_name)
| fieldsAdd affected_count = arraySize(affected_entity_ids)
| fieldsAdd bucket = if(affected_count &lt;= 1, "single-entity (expected)", else: "multi-entity (audit for topology gap)")
| summarize problem_count = count(), by:{bucket}
| sort problem_count desc
```

```
// RCA attachment rate — % of non-duplicate problems with a populated root cause, trended daily
fetch dt.davis.problems, from:-30d
| filter not(dt.davis.is_duplicate)
| fieldsAdd rca_flag = if(isNotNull(root_cause_entity_id), 100.0, else: 0.0)
| makeTimeseries rca_attachment_pct = avg(rca_flag), interval:1d
```

9. Cross-Series Pointers

- **WFLOW-04** — wire active problems into notification workflows
- **DASH-05** — problem-driven SLO and executive dashboards
- **AIOPS-04** — Dynatrace Assist generates problem-summary narratives via Generative AI
- **AIOPS-06** — Workflow tutorial: Summarize open problems with AI
